

Universidad de las Fuerzas Armadas - ESPE
Departamento de Ciencias de la Computación
Carrera de Ingeniería de Software



Análisis y Diseño de Software - NRC: 30723

Patrones Estructurales: Composite y Proxy

Autores:

Cedeño Andrés

Santi Jeancarlo

Tufiño Erick

Profesor: Ing. Jenny Ruiz

Fecha: 21 de Abril de 2026

1. Patrón estructural Composite:	3
1.1. Definición	3
1.2. Características	3
1.3. Ventajas y desventajas	3
1.4. Modelo de clases	4
1.5. Ejemplo Implementado en Código	5
1.6. ¿Por qué usar el patrón Composite?	6
2. Patrón estructural Proxy:	7
2.1. Definición	7
2.2. Características:	7
2.3. Ventajas y desventajas	7
2.4. Modelo de clases	8
2.5. Ejemplo Implementado en Código	11
2.6. ¿Por qué usar el patrón proxy?	11

1. Patrón estructural Composite:

1.1. Definición

Compone objetos en estructuras de árbol para representar jerarquías de parte-todo. Permite a los clientes tratar a los objetos individuales y a las composiciones de objetos de manera uniforme

1.2. Características

- Se basa en la composición recursiva para organizar un número abierto de objetos de modo que se puedan tratar múltiples elementos como uno solo.
- La clave es una clase abstracta que representa tanto a los elementos primitivos como a sus contenedores, la cual declara las operaciones comunes a todos los objetos.
- Sus participantes son Component (interfaz), Leaf (objeto primitivo sin hijos), Composite (componente con hijos) y el Client

1.3. Ventajas y desventajas

Ventajas:

- Simplifica el código del cliente: Los clientes interactúan con componentes individuales y estructuras compuestas de la misma forma, lo cual evita que tengan que usar múltiples sentencias lógicas para averiguar si están tratando con una hoja o con un contenedor.
- Facilita la adición de nuevos componentes: Las nuevas subclases derivadas (hojas o compuestos) funcionan automáticamente con el código existente del cliente sin requerir modificaciones.

Desventajas:

- Hace el diseño excesivamente general: La desventaja de que sea fácil agregar componentes es que resulta muy difícil restringir qué elementos se permiten dentro de un compuesto.
- Dependencia en comprobaciones en tiempo de ejecución: No se puede confiar en el sistema de tipos para obligar ciertas restricciones entre los componentes, por lo que se deben usar comprobaciones manuales en tiempo de ejecución.

1.4. Modelo de clases

Código en PlantUml

```
@startuml
title Patrón Composite aplicado a CRUD de Estudiantes

interface ComponenteAcademico {
    + mostrarInformacion(): String
}

class Estudiante {
    - id: int
    - nombre: String
    - edad: int

    + Estudiante(id: int, nombre: String, edad: int)
    + getId(): int
    + getNombre(): String
    + getEdad(): int
    + setNombre(nombre: String): void
    + setEdad(edad: int): void
    + mostrarInformacion(): String
}

class GrupoEstudiantes {
    - nombreGrupo: String
    - componentes: List<ComponenteAcademico>

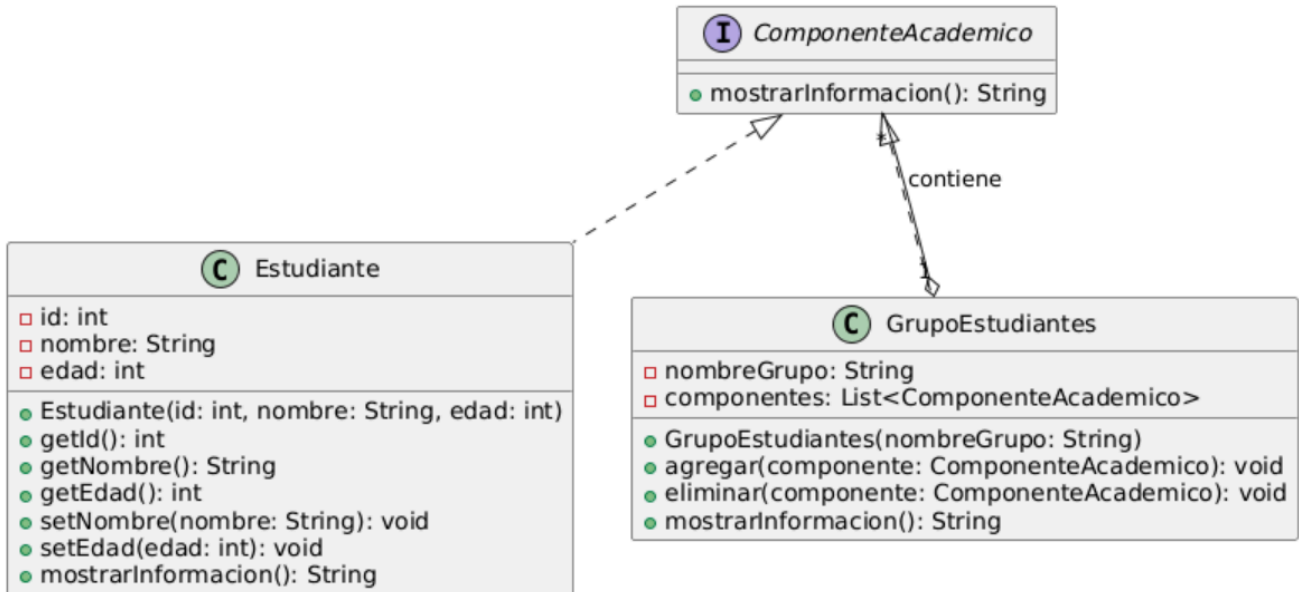
    + GrupoEstudiantes(nombreGrupo: String)
    + agregar(componente: ComponenteAcademico): void
    + eliminar(componente: ComponenteAcademico): void
    + mostrarInformacion(): String
}

ComponenteAcademico <|.. Estudiante
ComponenteAcademico <|.. GrupoEstudiantes

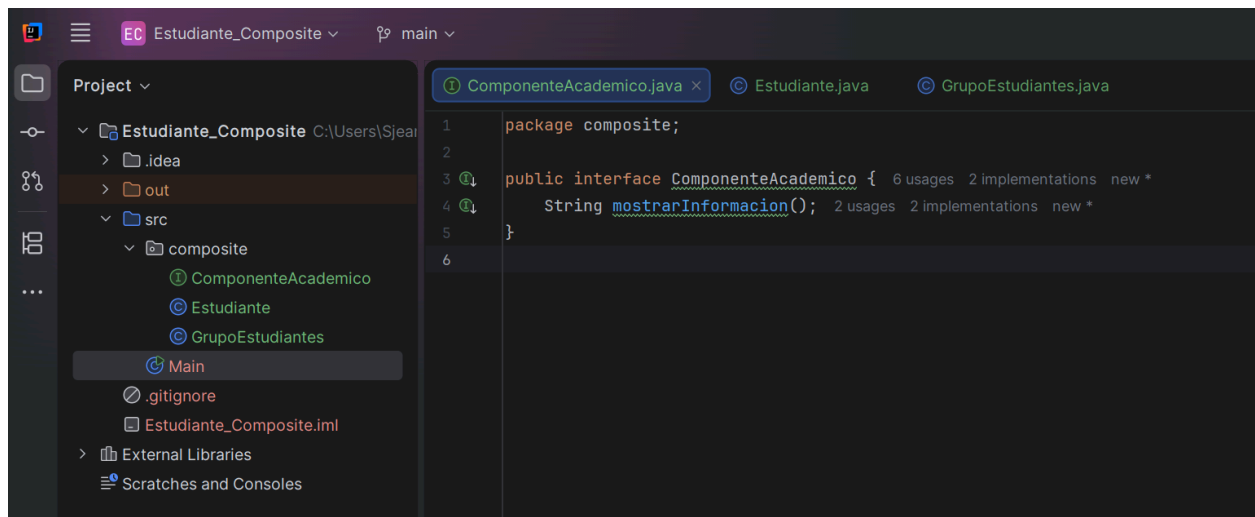
GrupoEstudiantes "1" o-- "*" ComponenteAcademico : contiene

@enduml
```

Patrón Composite aplicado a CRUD de Estudiantes



1.5. Ejemplo Implementado en Código



```

ComponenteAcademico.java
1 package composite;
2
3 public interface ComponenteAcademico {
4     String mostrarInformacion(); 2 usages
5 }

Estudiante.java
1 package composite;
2
3 public class Estudiante implements ComponenteAcademico {
4     private int id;
5     private String nombre;
6     private int edad;
7
8     public Estudiante(int id, String nombre, int edad) {
9         this.id = id;
10        this.nombre = nombre;
11        this.edad = edad;
12    }
13
14    public int getId() { return id; }
15
16    public String getNombre() { return nombre; }
17    public int getEdad() { return edad; }
18    public void setNombre(String nombre) { this.nombre = nombre; }
19    public void setEdad(int edad) { this.edad = edad; }
20
21    @Override
22    public String mostrarInformacion() {
23        return "Estudiante [ID: " + id + ", Nombre: " + nombre + ", Edad: " + edad + "]\n";
24    }
25 }

GrupoEstudiantes.java
1 package composite;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 public class GrupoEstudiantes implements ComponenteAcademico {
7     private String nombreGrupo;
8     private List<ComponenteAcademico> componentes;
9
10    public GrupoEstudiantes(String nombreGrupo) {
11        this.nombreGrupo = nombreGrupo;
12        this.componentes = new ArrayList<>();
13    }
14
15    public void agregar(ComponenteAcademico componente) {
16        componentes.add(componente);
17    }
18
19    public void eliminar(ComponenteAcademico componente) {
20        componentes.remove(componente);
21    }
22
23    @Override
24    public String mostrarInformacion() {
25        StringBuilder informacion = new StringBuilder();
26        informacion.append("Grupo: ").append(nombreGrupo).append("\n");
27
28        for (ComponenteAcademico componente : componentes) {
29            informacion.append(" - ")
30                .append(componente.mostrarInformacion())
31                .append("\n");
32        }
33
34        return informacion.toString();
35    }
36 }

Main.java
1 import composite.Estudiante;
2 import composite.GrupoEstudiantes;
3
4 public class Main {
5     public static void main(String[] args) {
6
7         Estudiante estudiante1 = new Estudiante(1, "Jencarlo", 15);
8         Estudiante estudiante2 = new Estudiante(2, "Ana", 18);
9         Estudiante estudiante3 = new Estudiante(3, "Luis", 20);
10
11         GrupoEstudiantes grupoPrincipal = new GrupoEstudiantes("Sexto Semestre");
12
13         grupoPrincipal.agregar(estudiante1);
14         grupoPrincipal.agregar(estudiante2);
15         grupoPrincipal.agregar(estudiante3);
16
17         System.out.println(grupoPrincipal.mostrarInformacion());
18     }
19 }

```

1.6. ¿Por qué usar el patrón Composite?

Consideramos que el patrón composite se usa para poder construir una estructura jerárquica por ejemplo de tipo árbol que permite tratar por igual a objetos individuales y a grupos de objetos. Esto elimina la necesidad de realizar condicionales para saber si un elemento es simple o si es un contenedor.

Elemento del patrón Composite	Clase en el proyecto	Responsabilidad
Component / Componente	ComponenteAcademico	Esta define la interfaz común del patrón. Establece que todo componente académico debe implementar <code>mostrarInformacion()</code>

Leaf / Hoja	Estudiante	Esta clase representa el objeto individual de la estructura. Guarda los datos del estudiante como: id, nombre y edad. No contiene otros componentes
Composite / Compuesto	GrupoEstudiantes	Esta clase representa el objeto compuesto. Contiene una lista de ComponenteAcademico, por lo que puede agrupar estudiantes u otros grupos y mostrar toda su estructura.
Cliente / Ejecución	Main	Aquí creamos estudiantes, creamos un grupo, se agregan estudiantes al grupo y se ejecuta mostrarInformacion para comprobar el funcionamiento del patrón.

2. Patrón estructural Proxy:

2.1. Definición

El patrón Proxy es un patrón estructural que utiliza un objeto intermediario para controlar el acceso a otro objeto real. En lugar de que el cliente use directamente el objeto real, primero pasa por el proxy, que puede validar, proteger, controlar, registrar o retrasar la operación antes de delegarla.

2.2. Características:

- Está pensado para retrasar el acceso a un objeto con el fin de ahorrar los recursos que implica la creación del objeto o su instanciación, solo dándole acceso cuando sea estrictamente necesario.
- Se recurre a él cuando los objetos son demasiado grandes (por ejemplo imágenes) en vez de completarlas todas a la vez se usa un PROXY que

actúa como un sustituto de la imagen real y se encarga de crearla solo cuando es necesario

2.3. Ventajas y desventajas

Ventajas:

- Oculta a los clientes si un objeto está en otro espacio de memoria (proxy remoto).
- Ahorra recursos y tiempo mediante optimizaciones como crear objetos costosos por encargo o utilizar la técnica de "copia-de-escritura" (proxy virtual).
- Añade de forma transparente tareas de mantenimiento o seguridad (proxies de protección o inteligentes).

Desventajas:

- Si se implementan para instanciar el objeto real, algunos proxies virtuales deben conocer explícitamente la clase concreta del SujetoReal que instancian.
- Pueden existir sobrecostos ocultos o complejidades dependiendo de las características del lenguaje, como la necesidad de referirse a identificadores de objetos independientes del espacio de memoria si no están creados.

2.4. Modelo de clases

Código en PlantUml

```
@startuml
title Patrón Proxy aplicado a CRUD de Estudiantes

class Estudiante {
    - id: int
    - nombre: String
    - edad: int

    + Estudiante(id: int, nombre: String, edad: int)
    + getId(): int
    + getNombre(): String
    + getEdad(): int
    + setNombre(nombre: String): void
    + setEdad(edad: int): void
}
```



```

    + toString(): String
}

interface IEstudianteRepository {
    + agregar(estudiante: Estudiante): boolean
    + actualizar(estudiante: Estudiante): boolean
    + eliminar(id: int): boolean
    + mostrarTodos(): List<Estudiante>
    + buscarPorId(id: int): Estudiante
}

class EstudianteRepositoryReal {
    - estudiantes: List<Estudiante>

    + agregar(estudiante: Estudiante): boolean
    + actualizar(estudiante: Estudiante): boolean
    + eliminar(id: int): boolean
    + mostrarTodos(): List<Estudiante>
    + buscarPorId(id: int): Estudiante
}

class EstudianteRepositoryProxy {
    - repositorioReal: EstudianteRepositoryReal

    + EstudianteRepositoryProxy(repositorioReal: EstudianteRepositoryReal)
    + agregar(estudiante: Estudiante): boolean
    + actualizar(estudiante: Estudiante): boolean
    + eliminar(id: int): boolean
    + mostrarTodos(): List<Estudiante>
    + buscarPorId(id: int): Estudiante
    - validarEstudiante(estudiante: Estudiante): boolean
}

class EstudianteController {
    - repositorio: IEstudianteRepository

    + EstudianteController(repositorio: IEstudianteRepository)
    + agregarEstudiante(id: int, nombre: String, edad: int): boolean
    + actualizarEstudiante(id: int, nombre: String, edad: int): boolean
    + eliminarEstudiante(id: int): boolean
    + mostrarTodos(): List<Estudiante>
}

class EstudianteView {
    - controller: EstudianteController

    + mostrarFormulario(): void
    + mostrarEstudiantes(): void
}

```

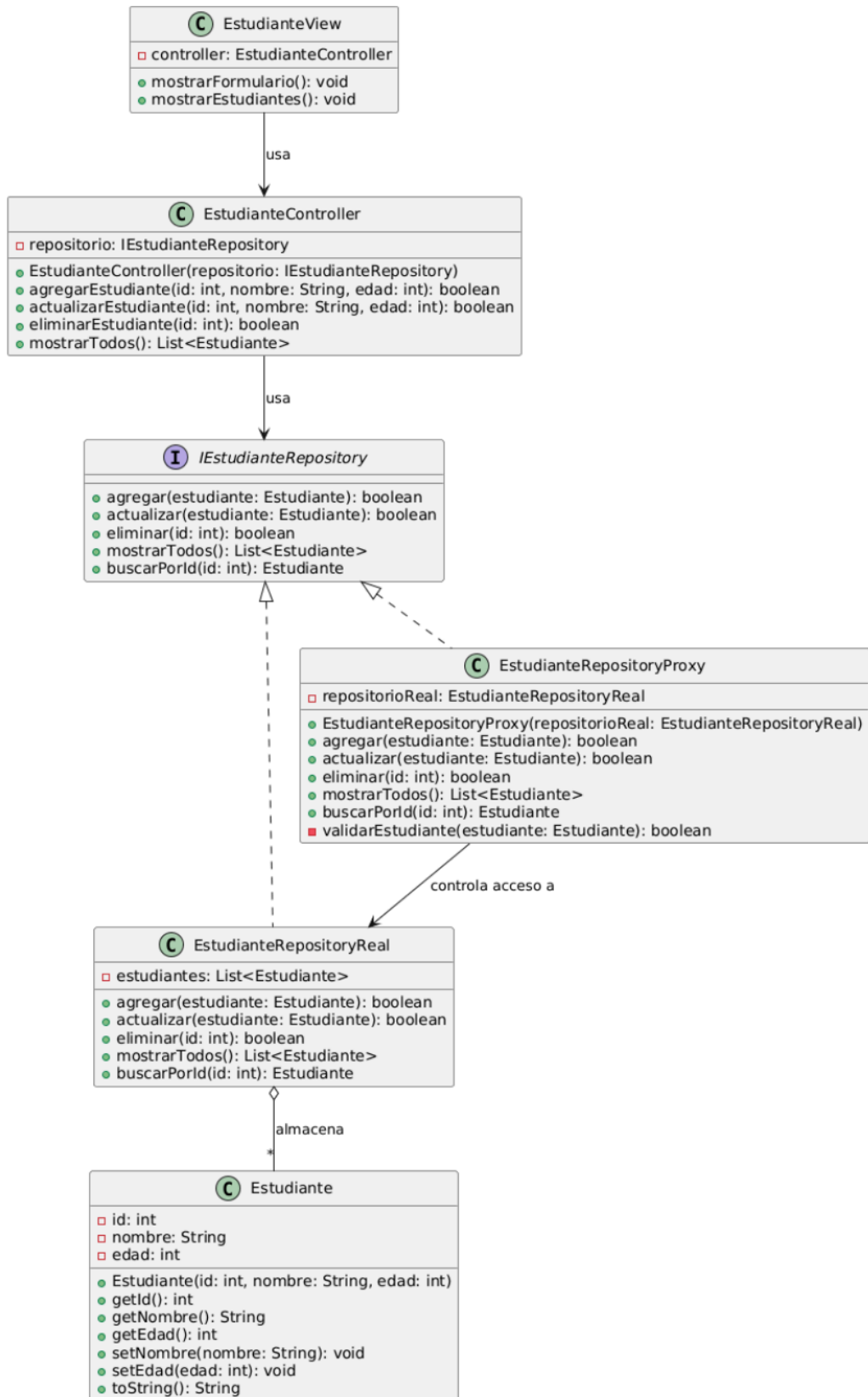
```
IEstudianteRepository <|.. EstudianteRepositoryReal  
IEstudianteRepository <|.. EstudianteRepositoryProxy
```

```
EstudianteRepositoryProxy --> EstudianteRepositoryReal : controla acceso a  
EstudianteController --> IEstudianteRepository : usa  
EstudianteView --> EstudianteController : usa
```

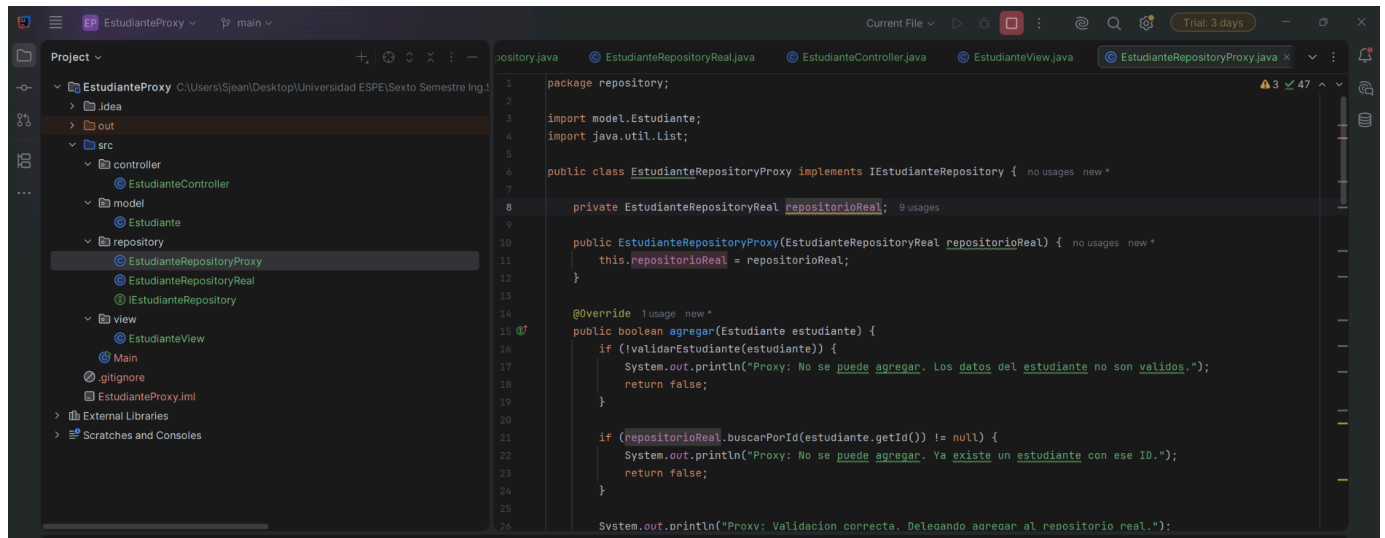
```
EstudianteRepositoryReal o-- "*" Estudiante : almacena
```

```
@enduml
```

Patrón Proxy aplicado a CRUD de Estudiantes

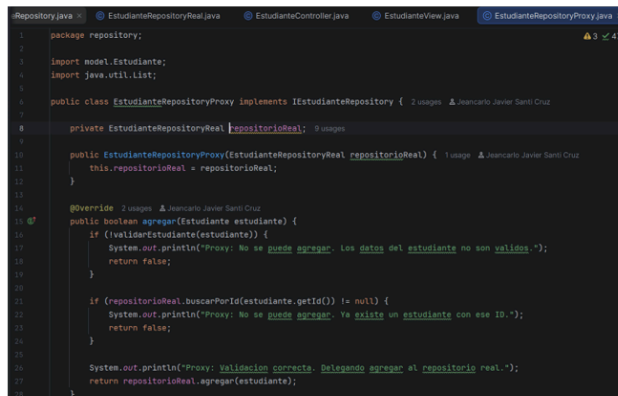


2.5. Ejemplo Implementado en Código



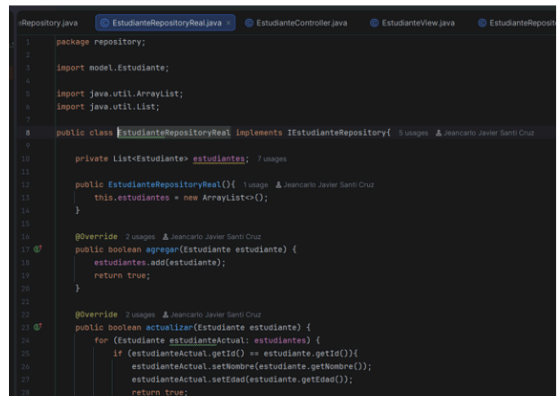
The screenshot shows an IDE with a project named 'EstudianteProxy'. The left sidebar shows the project structure with folders 'idea', 'out', 'src', 'controller', 'model', 'repository', and 'view'. The 'repository' folder is expanded, showing 'EstudianteRepositoryProxy', 'EstudianteRepositoryReal', and 'IEstudianteRepository'. The main editor shows the implementation of 'EstudianteRepositoryProxy' in 'EstudianteRepositoryProxy.java'.

```
1 package repository;
2
3 import model.Estudiante;
4 import java.util.List;
5
6 public class EstudianteRepositoryProxy implements IEstudianteRepository { no usages new *
7
8     private EstudianteRepositoryReal repositorioReal; 9 usages
9
10    public EstudianteRepositoryProxy(EstudianteRepositoryReal repositorioReal) { no usages new *
11        this.repositorioReal = repositorioReal;
12    }
13
14    @Override 1 usage new *
15    public boolean agregar(Estudiante estudiante) {
16        if (!validarEstudiante(estudiante)) {
17            System.out.println("Proxy: No se puede agregar. Los datos del estudiante no son validos.");
18            return false;
19        }
20
21        if (repositorioReal.buscarPorId(estudiante.getId()) != null) {
22            System.out.println("Proxy: No se puede agregar. Ya existe un estudiante con ese ID.");
23            return false;
24        }
25
26        System.out.println("Proxy: Validacion correcta. Delegando agregar al repositorio real.");
```



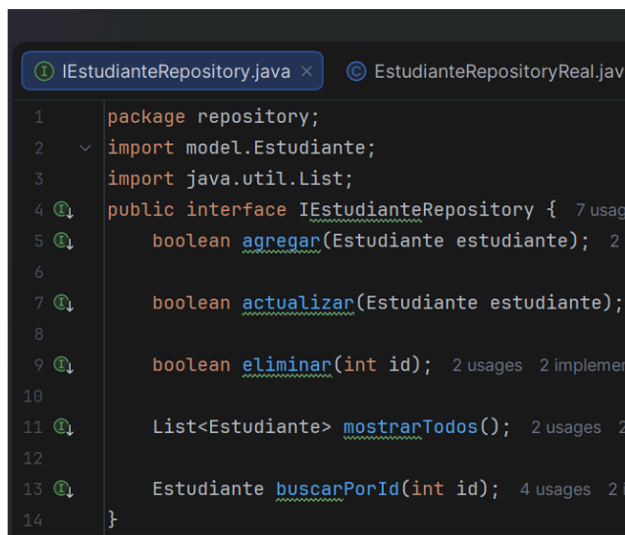
This screenshot shows the implementation of the 'agregar' method in 'EstudianteRepositoryProxy.java'. It includes validation logic and delegates the call to the 'repositorioReal'.

```
1 package repository;
2
3 import model.Estudiante;
4 import java.util.List;
5
6 public class EstudianteRepositoryProxy implements IEstudianteRepository { 2 usages 1 Jeancarlo Javier Santi Cruz
7
8     private EstudianteRepositoryReal repositorioReal; 9 usages
9
10    public EstudianteRepositoryProxy(EstudianteRepositoryReal repositorioReal) { 1 usage 1 Jeancarlo Javier Santi Cruz
11        this.repositorioReal = repositorioReal;
12    }
13
14    @Override 2 usages 1 Jeancarlo Javier Santi Cruz
15    public boolean agregar(Estudiante estudiante) {
16        if (!validarEstudiante(estudiante)) {
17            System.out.println("Proxy: No se puede agregar. Los datos del estudiante no son validos.");
18            return false;
19        }
20
21        if (repositorioReal.buscarPorId(estudiante.getId()) != null) {
22            System.out.println("Proxy: No se puede agregar. Ya existe un estudiante con ese ID.");
23            return false;
24        }
25
26        System.out.println("Proxy: Validacion correcta. Delegando agregar al repositorio real.");
27        return repositorioReal.agregar(estudiante);
28    }
29 }
```



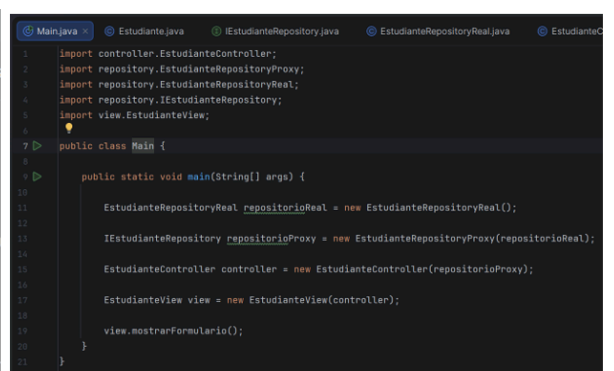
This screenshot shows the implementation of the 'agregar' method in 'EstudianteRepositoryReal.java'. It adds the student to a list and returns true.

```
1 package repository;
2
3 import model.Estudiante;
4 import java.util.ArrayList;
5 import java.util.List;
6
7 public class EstudianteRepositoryReal implements IEstudianteRepository { 5 usages 1 Jeancarlo Javier Santi Cruz
8
9     private List<Estudiante> estudiantes; 7 usages
10
11    public EstudianteRepositoryReal() { 1 usage 1 Jeancarlo Javier Santi Cruz
12        this.estudiantes = new ArrayList<>();
13    }
14
15    @Override 2 usages 1 Jeancarlo Javier Santi Cruz
16    public boolean agregar(Estudiante estudiante) {
17        estudiantes.add(estudiante);
18        return true;
19    }
20
21    @Override 2 usages 1 Jeancarlo Javier Santi Cruz
22    public boolean actualizar(Estudiante estudiante) {
23        for (Estudiante estudianteActual: estudiantes) {
24            if (estudianteActual.getId() == estudiante.getId()) {
25                estudianteActual.setNombre(estudiante.getNombre());
26                estudianteActual.setEdad(estudiante.getEdad());
27                return true;
28            }
29        }
30        return false;
31    }
32 }
```



This screenshot shows the definition of the 'IEstudianteRepository' interface in 'IEstudianteRepository.java'. It defines methods for adding, updating, deleting, and listing students.

```
1 package repository;
2
3 import model.Estudiante;
4 import java.util.List;
5
6 public interface IEstudianteRepository { 7 usages
7
8     boolean agregar(Estudiante estudiante); 2 usages
9
10    boolean actualizar(Estudiante estudiante);
11
12    boolean eliminar(int id); 2 usages 2 implement
13
14    List<Estudiante> mostrarTodos(); 2 usages 2 implement
15
16    Estudiante buscarPorId(int id); 4 usages 2 implement
17 }
```



This screenshot shows the 'Main' class in 'Main.java'. It initializes the repository, controller, and view, and calls the 'mostrarFormulario' method.

```
1 import controller.EstudianteController;
2 import repository.EstudianteRepositoryProxy;
3 import repository.EstudianteRepositoryReal;
4 import repository.IEstudianteRepository;
5 import view.EstudianteView;
6
7 public class Main {
8
9     public static void main(String[] args) {
10
11        EstudianteRepositoryReal repositorioReal = new EstudianteRepositoryReal();
12        IEstudianteRepository repositorioProxy = new EstudianteRepositoryProxy(repositorioReal);
13
14        EstudianteController controller = new EstudianteController(repositorioProxy);
15
16        EstudianteView view = new EstudianteView(controller);
17
18        view.mostrarFormulario();
19    }
20 }
```

2.6. ¿Por qué usar el patrón proxy?

Nosotros opinamos que el patrón proxy nos permite poner un intermediario entre quien está solicitando una operación y el objeto real que la ejecuta. Por ejemplo, en el código realizado del CRUD de Estudiante, nos sirvió para validar datos, evitar IDs repetidos, impedir actualizaciones o eliminaciones inválidas y recién después delegar la operación al repositorio real.

Elemento del patrón Proxy	Clase en el proyecto	Responsabilidad
Interfaz común / Subject	IEstudianteRepository	Esta interfaz define las operaciones que se deben cumplir tanto en el proxy como en el objeto real.
Objeto real / RealSubject	EstudianteRepositoryReal	Esta clase ejecuta realmente las operaciones CRUD sobre la lista de estudiantes
Proxy	EstudianteRepositoryProxy	Esta clase es la más importante ya que es el corazón del patrón, controla el acceso al repositorio real, valida las operaciones y delega si todo es correcto.
Objeto de datos	Estudiante	Esta clase representa la información del estudiante que será administrada por nuestro repositorio.
Cliente / Ejecución	Main	Crea y conecta el repositorio real con el proxy para iniciar el correcto uso del patrón